

Exercice d'apprentissage pour ECS

"Artefacts Survivors"

Setup global

1. Setup une nouvelle scène avec un sol centré en 0,0,0 et un GameObject `DataManager`
2. Créer une class `DataManager` héritant de `MonoBehaviour` qui instancie un `EcsWorld` au Start et le détruit correctement au Destroy. L'ajouter dans la scène sur le GameObject correspondant.

Spawn des entités et les déplacer avec les inputs joueur

3. Créer une struct `VisualComponent` avec une variable "public GameObject Instance"
4. Créer une class `GameInitSystem` qui implémente `IEcsInitSystem` et crée une nouvelle entité, l'ajoute à la pool `VisualComponent`, et initialise son `VisualComponent.Instance` en créant une capsule (GameObject Unity).
5. Dans le `DataManager.Start`, lancer une coroutine d'initialisation (voir exemples). Ajouter la class `GameInitSystem` au `EcsSystems` du `DataManager` dans la coroutine d'init. Appeler ensuite le `EcsSystems.Init` à la fin de la coroutine et le `Run` dans `DataManager.Update` (voir exemples)
6. Créer une class `InputManagerSystem` qui implémente `IEcsRunSystem` et dans le `Run` va itérer sur un filtre incluant le component `VisualComponent`. L'ajouter au `DataManager`.
7. Dans le `Run`, en fonction de l'input joueur ZQSD par ex. (utiliser `Input.GetKey(KeyCode.XXX)`), déplacer le GameObject en utilisant `VisualComponent.Instance.transform.SetPositionAndRotation()`.

Caméra

8. Créer un système qui update la position de la caméra et qui se déplace pour toujours placer le joueur au centre de l'écran.

Créer les ennemis

9. Dans le `GameInitSystem.Init`, faire une boucle (sur un nombre configurable depuis le `DataManager`) et créer des nouvelles entités qui seront les ennemis. Leur ajouter également le `VisualComponent` et leur faire spawn une capsule de couleur différente de celle du joueur. Faire

en sorte que leur position soit aléatoire sur la plateforme (éloigné du joueur qui devrait être en 0,0)

Différencier joueur et ennemis

A ce stade, les inputs vont déplacer toutes les capsules en même temps. Il faut les différencier :

10. Créer un `PlayerTagComponent` vide et ajouter l'entité du player à la Pool. Créer un `EnemyTagComponent` vide et l'ajouter aux entités des ennemis.

11. Dans le `InputManagerSystem`, modifier le filtre pour inclure le `PlayerTagComponent` : les inputs ne déplacent alors que la capsule du joueur.

Faire bouger les ennemis vers le player

12. Créer une classe `EnemyMovementSystem` qui va itérer sur un filtre incluant `VisualComponent` et `EnemyTagComponent`. Dans ce jeu les ennemis vont se déplacer vers le joueur. Il faut donc récupérer la position du joueur, et venir modifier `VisualComponent.Instance.transform` des ennemis en les déplaçant progressivement vers le joueur (utiliser `Time.deltaTime` et `Unity.Mathematics.math.lerp` par exemple)

Le moyen de se défendre : les bullets

Pour pouvoir communiquer entre eux, les systèmes ne peuvent pas utiliser de callbacks ou de delegate comme dans la POO classique. On utilise donc des entités avec un component spécifique pour créer un "évènement" qui sera écouté par les autres systèmes. Ici on va créer un event correspondant au spawn des bullets.

13. Dans `InputManagerSystem`, créer une fonction `void` qui crée une entité d'event et lui ajoute un nouveau component `SpawnBulletEventComponent`. Appeler cette fonction dans le `Run` après l'appui sur l'input choisi (barre espace par ex.).

14. Créer une classe `BulletMovementSystem` qui implémente `IEcsRunSystem`. Créer une fonction qui crée une entité et lui ajoute le component `VisualComponent` (le visuel peut être un cube par exemple) puis la retourne. Créer un nouveau component `SpeedComponent` avec une variable `public float Speed`. Faire une variable configurable dans `DataManager` `m_bulletSpeed`. Ajouter le component `SpeedComponent` à l'entité de bullet dans la fonction et setup sa variable `Speed` avant de renvoyer l'entité.

15. Créer un `BulletComponent` qui a en paramètre un `public float3 Direction`. Ajouter le `BulletComponent` à l'entité dans la fonction précédente en initialisant la `Direction` avec une valeur aléatoires pour les axes x et z.

16. Dans le `Run`, faire une première boucle qui va itérer sur un filtre de `SpawnBulletEventComponent` et appeler la fonction de création de la bullet.

17. Dans une seconde boucle, itérer sur un filtre `VisualComponent && BulletComponent` et faire se déplacer la bullet dans la direction stockée dans le `BulletComponent`, avec la vitesse stockée dans le `SpeedComponent`.

A ce stade, les component d'events sont créés, mais jamais détruits. En lançant le jeu, le nombre de bullets créées ne va faire qu'augmenter exponentiellement. Il faut donc "Clear" les events une fois consommés

18. Solution : dans une troisième boucle, itérer sur un filtre de `SpawnBulletEventComponent` et appeler `world.DelEntity(eventEntity)`.

19. Ajouter la notion de "LifeTime" sur les bullets pour qu'elles se détruisent toutes seules quand elles sont trop loin du joueur.

20. Implémenter la détection de la collision. Dans un premier temps, si la bullet collisionne avec un ennemi, on détruit l'ennemi.

21. Trouver un moyen de détruire aussi le visuel de l'ennemi et pas uniquement son entité.

22. Ajouter la notion de points de vie aux ennemis et de dégâts sur les bullets configurables depuis le `DataManager`. A la collision de la bullet, faire descendre les points de vie de l'ennemi. Conditionner la destruction de l'ennemi à ses points de vie tombant à 0.

But du jeu : survie

23. Ajouter la notion de points de vie sur le joueur (configurable sur le `DataManager`). Lorsque l'ennemi touche le joueur, lui enlever un montant de dégâts (configurable sur le `DataManager`) et détruire l'ennemi.

24. Afficher les points de vie restants à l'écran au format "X/Y" X étant la vie courante et Y la vie max.

25. Lorsque le joueur atteint 0 afficher le GameOver (faire un bouton replay qui va reset les infos du world)

Pistes d'amélioration

26. Utiliser un `TransformComponent` pour update la position plutôt que de set la transform directement sur le gameobject. Trouver une méthode pour mettre à jour la position des `GameObject` à partir de l'information de ce component.

27. Augmenter le nombre de bullets et/ou les dégâts des bullets en fonction du nombre d'ennemis tués par le joueur (ne pas compter ceux qui ont été détruits en faisant des dégâts au joueur)

28. Augmenter le nombre d'ennemis et/ou leur vitesse en fonction du temps de jeu

29. Modifier les différents systèmes pour ajouter des jobs Unity

(<https://docs.unity3d.com/Manual/JobSystem.html>)